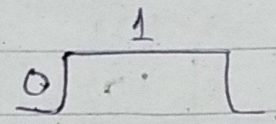
Thursday
16 Jan, 2025

BINARY / DIGITAL FUNCTION

→ Descrete function (hai ya nh hai)

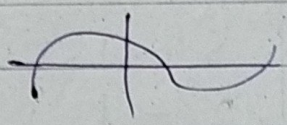


→ Any data written in high-level language (easily understandable by humans) is in binary form in computers.

→ ASCII
only form to convert alphabets into binary.

CONTINUOUS / ANALOGUE FUNCTION

→ sinusodial graph
→ change at every point



- BINARY SYSTEM: $(1, 0)_2$ → Low level language
→ defined by Boolean.
- DECIMAL SYSTEM: $(\ )_{10}$ $(0-\infty)$

- OCTAL SYSTEM: $(0-7)_8$

- HEXADECIMAL SYSTEM: $(0-15)_{16}$
10 - A, 11 - B, 12 - C, 13 - D, 14 - E, 15 - F

→ $(72F)_{10}$, 72F => both are not any system
→ $(72F)_{16}$ → a system

Softwares systems:
"A software that needs to run a hardware".
→ GUI (Graphical User Interface) → icons (by clicking)
↳ pictorial resolation increasing day by day (pixels) ....

① DECIMAL → BINARY

i) 723

2	723
2	361 – 1
2	180 – 1
2	90 – 0
2	45 – 0
2	22 – 1
2	11 – 0
2	5 – 1
2	2 – 1
	1 – 0

$(723)_{10} = (1011001011)_2$

ii) 1024

2	1024
2	512 – 0
2	256 – 0
2	128 – 0
2	64 – 0
2	32 – 0
2	16 – 0
2	8 – 0
2	4 – 0
2	2 – 0
	1 – 0

$(1024)_{10} = (10000000000)_2$

② DECIMAL → OCTAL

i) 2647

8	2647
8	330 – 7
8	41 – 2
	5 – 1

$(2647)_{10} = (5127)_8$

ii) 4782

8	4782
8	597 – 6
8	74 – 5
8	9 – 2
	1 – 1

$(4782)_{10} = (11256)_8$

$8^2 = 64,\ 8^3 = 512,\ 8^4 = 4096,\ 8^5 = 32768$

$16^2 = 256,\ 16^3 = 4096,\ 16^4 = 65536,\ 16^5 = 1048576$

③ DECIMAL → HEXA

i) 10962

16	10962
16	685 – 2
16	42 – D
16	2 – A

$(10962)_{10} = (2AD2)_{16}$

ii) 24981

16	24981
16	1561 – 5
16	97 – 9
	6 – 1

$(24981)_{10} = (6195)_{16}$

④ BINARY → DECIMAL

i) $(10011)_2$

$\Rightarrow 1 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$

$\Rightarrow 1 \times 16 + 0 \times 8 + 0 \times 4 + 1 \times 2 + 1 \times 1$

$\Rightarrow 16 + 0 + 0 + 2 + 1 \Rightarrow 19 \quad (10011)_2 = (19)_{10}$

⑤ OCTAL → DECIMAL

i) $(546)_8 = (358)_{10}$

$\Rightarrow 5 \times 8^2 + 4 \times 8^1 + 6 \times 8^0 = 5 \times 64 + 4 \times 8 + 6 \times 1$

$\Rightarrow 320 + 32 + 6 = 358$

⑥ HEX → DECIMAL

i) $(3AC8)_{16} = (15048)_{10}$

$\Rightarrow 3 \times 16^3 + A \times 16^2 + C \times 16^1 + 8 \times 16^0$

$= 3 \times 4096 + 10 \times 256 + 12 \times 16 + 8 \times 1$

$= 12288 + 2560 + 192 + 8 = 15048$

• Stimulator

1024 512 256 128 64 32 16 8 4 2 1

1 0 0 1 1

(16 + 2 + 1 = 19)

0 → (8, 4) ←

⑦ BINARY → OCTAL (triplets) → Right to left

i) $(\underline{010}\underline{111}\underline{010}\underline{001}\underline{111})_2$

2 7 2 1 7

$(27217)_8$

2 7 2 1 7

010 111 010 001 111

$(010111010001111)_2$

⑧ OCTAL → BINARY

$(27217)_8$

2 7 2 1 7

010 111 010 001 111

→ $(010111010001111)_2$

⑨ BINARY → HEXADECIMAL

$(\underline{0011}\underline{1010}\underline{0001}\underline{1011}\underline{1000})_2$

3 A 1 B 8

→ $(3A1B8)_{16}$

⑩ HEXADECIMAL → BINARY

$(3A1B8)_{16}$

(4 2 1)

0	0 0 0
1	0 0 1
2	0 1 0
3	0 1 1
4	1 0 0
5	1 0 1
6	1 1 0
7	1 1 1

8 4 2 1

3 A 1 B 8

0011 1010 0001 1011 1000

→ $(0011101000011011000)_{16}$

⑪ HEXADECIMAL → OCTAL

8 4 2 1

$(AC3)_{16}$

$(101011000011)_2 \rightarrow (5303)_8$

→ First we convert into binary then to octal.

⑫ OCTAL → HEXADECIMAL

BOOLEAN ALGEBRA:-

It is a convenient and systematic way of expressing and analyzing the operation of a logic circuit.

In 1850s an English mathematician Lan George Boole develop a mathematical system for formulating logical statement with symbols.

The term logic is applied to a digital circuit used to implement logical functions.

BOOLEAN ALGBRIC TERMS :-

1) Boolean constants: 0 and 1
2) Variables: Represents boolean constants. can be any alphabet from A to Z and a to z.
3) Complement: Inverse of a constant.

$A \Rightarrow \bar{A}$

$1 \Rightarrow 0$, $0 \Rightarrow 1$

TRUTH TABLE :-

A truth table is a systematic way of listing of values for dependent variable in terms of all possible values of independent variable.

In other words, it can also be defined as a table representing the conditions of input and output involving 2 or more variables.

LOGICAL OPERATORS :-

OR (+), AND (.), NOT (-)

1) OR :-

In any OR operator, if any input goes 1, output will be 1 else 0.

S.No	A	B	A + B
1	0	0	0
2	0	1	1
3	1	0	1
4	1	1	1

2) AND :-

In any AND operator, if all input goes 1, output will be 1 else 0.

S.No	A	B	A . B
1	0	0	0
2	0	1	0
3	1	0	0
4	1	1	1

3) NOT :- ($\bar{A}$, A')

NOT operator reverse the output.

S.No	A	$\bar{A}$
1	1	0
2	0	1

No. of rows in truth table :

Depends on number of variables $\Rightarrow$ no. of rows $= 2^{(n)}$ (n → no. of variables)

BOOLEAN EXPRESSION :-

A boolean expression is an arrangement of variables and logical operators used to express the operation of a logic circuit.

eg: $A + B = C \cdot B$

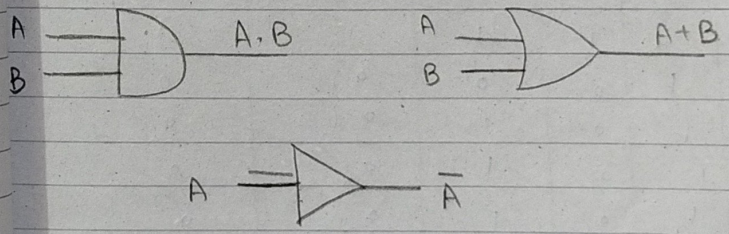
$A + \bar{B} \neq C + \bar{D}$

BOOLEAN FUNCTION :-

A boolean function is a function that takes binary inputs (0 or 1) and produces a binary output (0 or 1).

eg: $f(A, B) = A + B$

$W = A \cdot B \cdot C$



Q NO: Draw truth tables for the following boolean functi

i) $(A+B).\bar{C}$

S.NO	A	B	C	$\bar{C}$	A+B	$(A+B).\bar{C}$
1	0	0	0	1	0	0
2	0	1	0	1	1	1
3	1	0	0	1	1	1
4	1	1	0	1	1	1
5	0	0	1	0	0	0
6	0	1	1	0	1	0
7	1	0	1	0	1	0
8	1	1	1	0	1	0

ii) $[(x+y).\bar{z}].(x+y)$

S.NO	x	y	z	$\bar{z}$	x+y	$(x+y).\bar{z}$	$[(x+y).\bar{z}].(x+y)$
1	0	0	0	1	0	0	0
2	0	1	0	1	1	1	1
3	1	0	0	1	1	1	1
4	1	1	0	1	1	1	1
5	0	0	1	0	0	0	0
6	0	1	1	0	1	0	0
7	1	0	1	0	1	0	0
8	1	1	1	0	1	0	0

DE MORGAN'S LAW :-

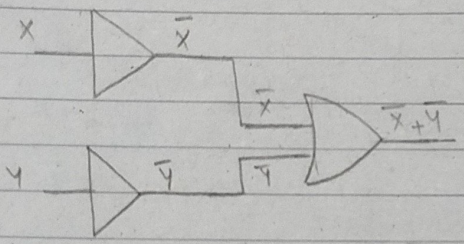
1) The complement of a product equals the sum of the complements.

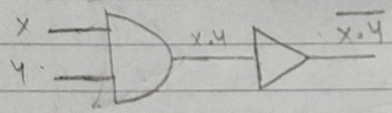
$$\overline{X.Y} = \overline{X} + \overline{Y}$$

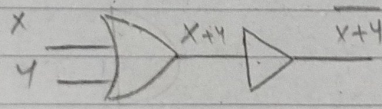
2) The complement of a sum equals the product of the complements

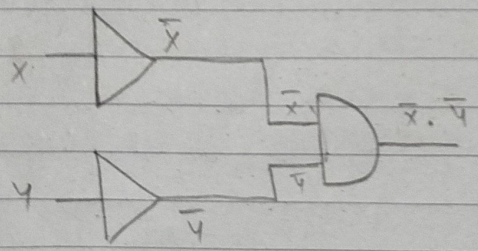
$$\overline{X+Y} = \overline{X}.\overline{Y}$$

* Break the bar, change the sign









S.NO	X	Y	$\overline{X}$	$\overline{Y}$	X.Y	$\overline{X.Y}$	$\overline{X}+\overline{Y}$
1	0	0	1	1	0	1	1
2	0	1	1	0	0	1	1
3	1	0	0	1	0	1	1
4	1	1	0	0	1	0	0

S.NO	X	Y	$\overline{X}$	$\overline{Y}$	X+Y	$\overline{X+Y}$	$\overline{X}.\overline{Y}$
1	0	0	1	1	0	1	1
2	0	1	1	0	1	0	0
3	1	0	0	1	1	0	0
4	1	1	0	0	1	0	0

LAWS OF BOOLEAN ALGEBRA :-

1) COMMUTATIVE LAW :-
   i) $A + B = B + A$
   ii) $A . B = B . A$

2) ASSOCIATIVE LAW :-
   i) $A + (B + C) = (A + B) + C$
   ii) $A(BC) = (AB)C$

3) DISTRIBUTIVE LAW :-
   i) $A(B + C) = AB + AC$
   ii) $AB + AC = A(B + C)$

RULES OF BOOLEAN ALGEBRA :-

i) $A + 0 = A$

ii) $A + 1 = 1$

iii) $A \cdot 0 = 0$

iv) $A \cdot 1 = A$

v) $A + A = A$

vi) $A \cdot A = A$

vii) $A + \bar{A} = 1$

viii) $A \cdot \bar{A} = 0$

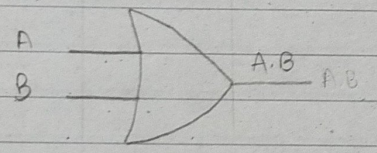
ix) $\bar{\bar{A}} = A$

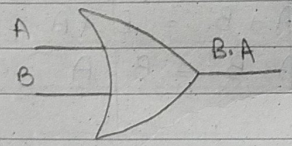
x) $A + A \cdot B = A$

xi) $A + \bar{A} \cdot B = A + B$

xii) $(A+B)(A+C) = A + BC$

A	B	A+B	B+A	A.B	B.A
0	0	0	0	0	0
0	1	1	1	0	0
1	0	1	1	0	0
1	1	1	1	1	1





1) Identity law:

$A + 0 = A$

$A \cdot 1 = A$

2) Dominance law:

$A + 1 = 1$

$A \cdot 0 = 0$

3) Idempotent law:

$A + A = A$

$A \cdot A = A$

4) Complement law:

$A + A' = 1$

$A \cdot A' = 0$

5) Involution law:

$(A')' = A$

6) Absorption law:

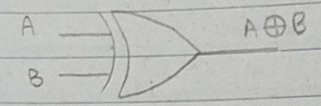
$A + (A \cdot B) = A$

$A \cdot (A + B) = A$

4) EX-OR :-

An Ex-OR gate gives a high output (1) only when the inputs are different.

- If both inputs are the same, the output is 0.
- If both inputs are different, the output is 1.



$A \oplus B = (\bar{A} \cdot B) + (A \cdot \bar{B})$

A	B	A⊕B
0	0	0
0	1	1
1	0	1
1	1	0

1) $A \oplus 0 = A$
2) $A \oplus 1 = \bar{A}$
3) $A \oplus A = 0$
4) $A \oplus B = B \oplus A$
5) $(A \oplus B) \oplus C = A \oplus (B \oplus$

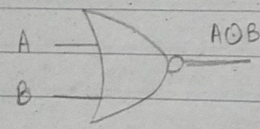
5) EX-NOR :-

An Ex-NOR gate gives high output if and only if the inputs are same.

- If both inputs are the same, the output is 1.
- If both inputs are the diff, the output is 0.

$A \odot B = \overline{(A \oplus B)} = A \cdot B + A' \cdot B'$

A	B	A⊙B
0	0	1
0	1	0
1	0	0
1	1	1



ADDER :-

An adder is a combinational logic circuit that performs binary addition.

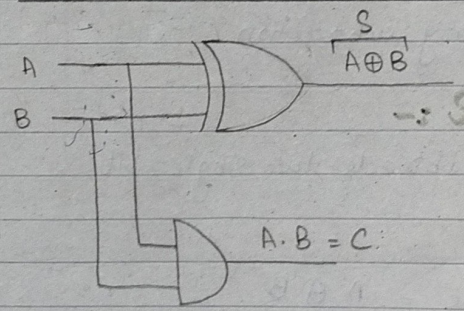
HALF ADDER :-

A half adder adds two single-bit binary numbers and gives a sum and a carry.

Inputs : A, B (1-bit each)
Outputs : Sum = $A \oplus B$
Carry = $A \cdot B$

3 - ⇒ 0 0 1 1 → 1 → carry
+ 5 0 1 0 1
8 0 1 1 0

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1



FULL ADDER :-

A full adder adds three 1-bit binary numbers :

→ A and B (the numbers)
→ Cin (Carry-in from previous bit).

Outputs : Sum = $A \oplus B \oplus Cin$

Carry = $(A.B) + (B.Cin) + (A.Cin)$

A	B	cin	Sum	Carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

SUBTRACTOR :-

A subtractor is a combinational circuit that performs binary subtraction.

HALF SUBTRACTOR :-

It subtracts two single-bit

inputs : A - B

Outputs : Difference = $A \oplus B$

Borrow = $\bar{A}.B$

circuit diagram

8 ⇒ 1 0 0 0
5 − 0 1 0 1
3 = 0 0 1 1

A	B	Difference	Borrow
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

FULL SUBTRACTOR :-

It subtracts :

A - B - Bin (Bin = Borrow-in)

Outputs :~ Difference = $A \oplus B \oplus Bin$

Borrow = $\bar{A}B + \bar{A}Bin + BBin$

A	B	Cin	Diff	Borrow
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

circuit diagram.

UNIVERSAL GATES :-

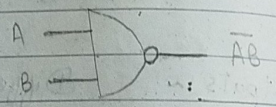
A universal gate is a logic gate that can be used to construct any other gate (AND, OR, NOT, XOR, etc.) and hence any digital logic circuit by using any gate type.

1) NAND GATE :-

A NAND Gate produces a low output only when all inputs are high. When any of the inputs is low, the output will be high.

→ Simply an AND gate followed by an INVERTOR.

A	B	$\overline{(A.B)}$
0	0	1
0	1	1
1	0	1
1	1	0

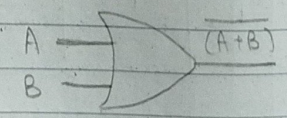


2) NOR GATE :-

A NOR Gate produces a low output when any of its inputs is high. Only when all of its input are low the output is high.

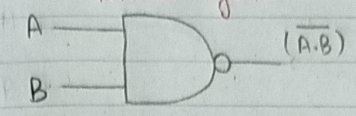
→ Simply OR gate followed by an INVERTOR

A	B	$\overline{(A+B)}$
0	0	1
0	1	0
1	0	0
1	1	0

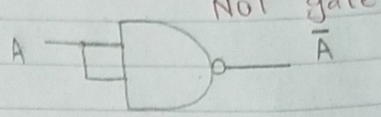


Implement all basic gates using NAND gate :-

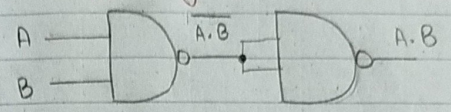
NAND gate



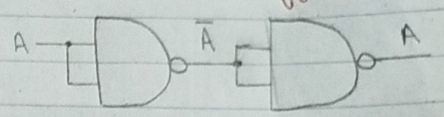
NOT gate



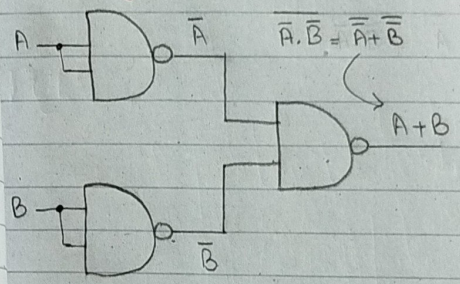
AND gate



Buffer



OR gate



$A.(\overline{A.B}) = \overline{A} + \overline{A\overline{B}} = \overline{A} + AB$

$B.(\overline{A.B}) = \overline{B} + \overline{A\overline{B}} = \overline{B} + AB$

$\overline{((\overline{A}+AB)(\overline{B}+AB))}$

$= \overline{A}\,\overline{B} + \underbrace{\overline{A}AB}_{0} + \underbrace{AB\overline{B}}_{0} + \underbrace{ABAB}_{AB}$

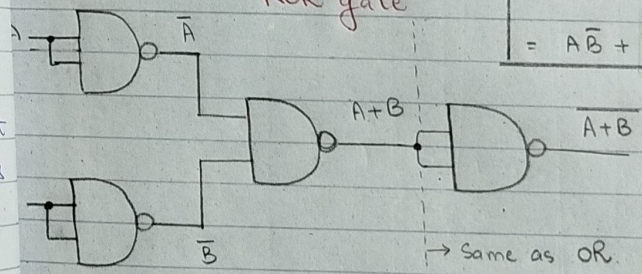
$= \overline{\overline{A}\,\overline{B} + AB} = \overline{(\overline{A}\,\overline{B})(\overline{AB})}$

$= (\overline{\overline{A}} + \overline{\overline{B}})(\overline{A}+\overline{B}) = (A+B)(\overline{A}+\overline{B})$

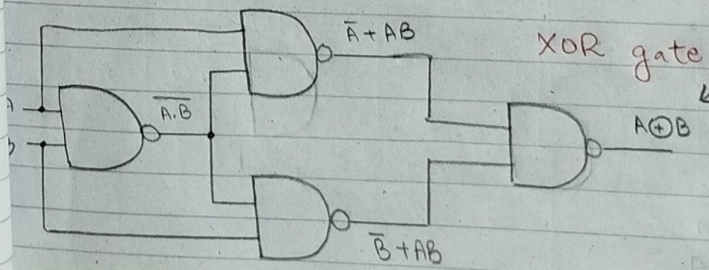
$= \underbrace{A\overline{A}}_{0} + A\overline{B} + \overline{A}B + \underbrace{B\overline{B}}_{0}$

$= A\overline{B} + \overline{A}B \Rightarrow A \oplus B$

NOR gate



XOR gate



STANDARD FORMS OF BOOLEAN EXPRESSION :-

All Boolean expressions, regardless of their forms, can be converted into either of two standard forms : the sum-of-products form or the product-of-sum form. Standardization makes the simplication, and implementation of boolean expressions must more easier & systematic.

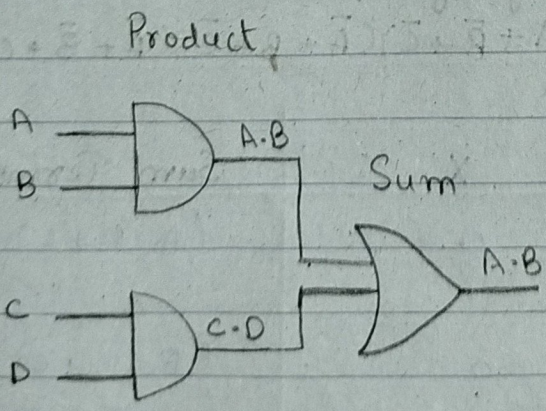
1) SUM OF PRODUCTS (SOP) :- (Derived minterms)

A Boolean expression written as a sum (OR) of product (AND) terms.

→ Each term is a product of literal (variables or their complements), and the terms are OR'ed together.

Examples : i) $AB + ABC$

ii) $\bar{A}B + \bar{A}B\bar{C} + AC$



$A + \bar{A} = 1$

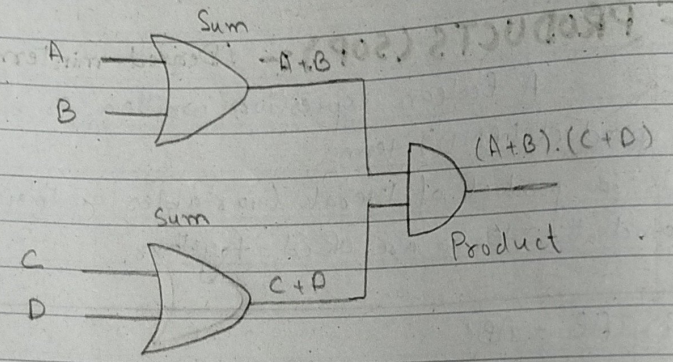
2) PRODUCT OF SUMS (POS) :- (Derived maxTerm)

A Boolean expression written as a product (AND) of sum (OR) terms.

→ Each term is a sum of literals (variables or complements) and the terms are AND'ed together.

Examples :- i) $(\bar{A}+B)(A+\bar{B}+C)$

ii) $(\bar{A}+\bar{B}+\bar{C})(C+\bar{D}+E)(\bar{B}+C+D)$



Determine the truth table for given standard POS expression

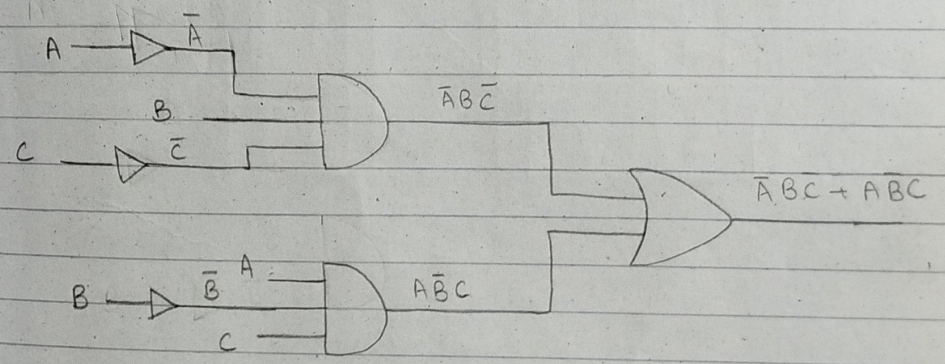
$(A+B+C)(A+\bar{B}+C)(A+\bar{B}+\bar{C})(\bar{A}+B+\bar{C})(\bar{A}+\bar{B}+C)$

A	B	C	X	Sum Terms
0	0	0	0	$(A+B+C)$
0	0	1	1	
0	1	0	0	$(A+\bar{B}+C)$
0	1	1	0	$(A+\bar{B}+\bar{C})$
1	0	0	1	
1	0	1	0	$(\bar{A}+B+\bar{C})$
1	1	0	0	$(\bar{A}+\bar{B}+C)$
1	1	1	1	

EX 4-20

RP : Create a truth table for the standard SoP expression $\bar{A}B\bar{C} + A\bar{B}C$.

INPUTS						EXPRESSION		OUTPUT
A	B	C	$\bar{A}$	$\bar{B}$	$\bar{C}$	$\bar{A}B\bar{C}$	$A\bar{B}C$	X
0	0	0	1	1	1	0	0	0
0	0	1	1	1	0	0	0	0
0	1	0	1	0	1	1	0	1
0	1	1	1	0	0	0	0	0
1	0	0	0	1	1	0	0	0
1	0	1	0	1	0	0	1	1
1	1	0	0	0	1	0	0	0
1	1	1	0	0	0	0	0	0



EX 4-21

RP: Develop a truth table for the following standard POS expression: $(A+\bar{B}+C)(A+B+\bar{C})(\bar{A}+\bar{B}+\bar{C})$

INPUTS						EXPRESSION			OUTPUT
A	B	C	$\bar{A}$	$\bar{B}$	$\bar{C}$	$A+\bar{B}+C$	$A+B+\bar{C}$	$\bar{A}+\bar{B}+\bar{C}$	X
0	0	0	1	1	1	1	1	1	1
0	0	1	1	1	0	1	0	1	0
0	1	0	1	0	1	0	1	1	0
0	1	1	1	0	0	1	1	1	1
1	0	0	0	1	1	1	1	1	1
1	0	1	0	1	0	1	1	1	1
1	1	0	0	0	1	1	1	1	1
1	1	1	0	0	0	1	1	0	0

THE KARNAUGH MAP (K-MAP) :-

A Karnaugh map (or K-map) is a visual method used to simplify Boolean algebra expressions without having to use Boolean laws and theorems.

→ It is similar to a truth table because it presents all of the possible values of input variables & the resulting output for each value.

→ The K-map is an array of cells in which each cell represent a binary value of the input variables

Total no. of cells => 2 is to the power total variables

e.g: For 3-variables: $2^3 = 8$
For 4-variables: $2^4 = 16$

→ K-map can be of 2 to 5 variables.

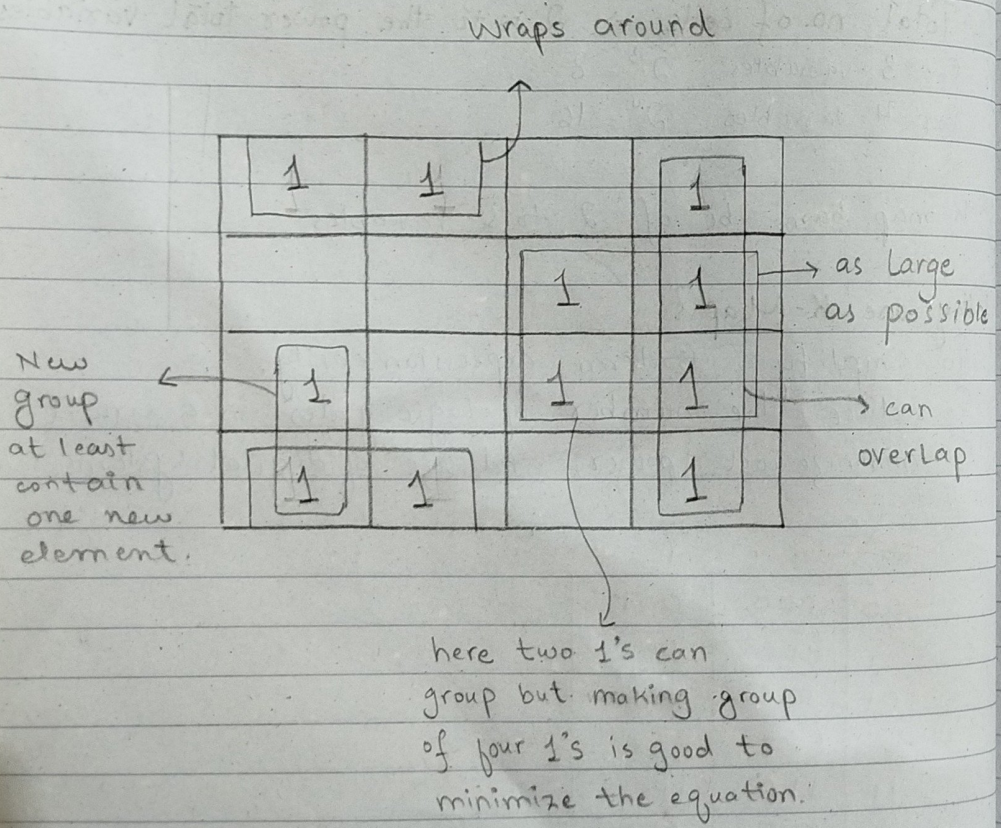
Why use K-Map?

i) To simplify Boolean expression easily.
ii) To reduce the number of logic gates in a circuit
iii) To minimize cost, power, and size of digital systems.

AB \ C	0	1
00	000 $\bar{A}\bar{B}\bar{C}$	001 $\bar{A}\bar{B}C$
01	010 $\bar{A}B\bar{C}$	011 $\bar{A}BC$
11	110 $AB\bar{C}$	111 ABC
10	100 $A\bar{B}\bar{C}$	101 $A\bar{B}C$

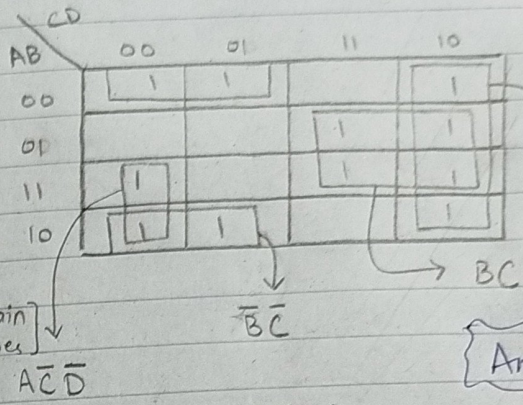
BASIC RULES:

1) Only adjacent cells can be grouped.
2) Groups must contain powers of 2 elements.
   (1, 2, 4, 8, ...)
3) Groups should be as large as possible.
4) Groups can overlap, but new group contain at least one new element
5) K-map wraps around (edges are adjacent)
6) Every 1 must be covered at least once.



SoP expression

Simplify $\bar{A}\bar{B}\bar{C}\bar{D} + \bar{A}\bar{B}\bar{C}D + \bar{A}\bar{B}C\bar{D} + \bar{A}BCD + \bar{A}BC\bar{D} + AB\bar{C}\bar{D} + ABCD + ABC\bar{D} + AB\bar{C}\bar{D} + A\bar{B}\bar{C}D + A\bar{B}CD$ using K-map



same remain
diff vanishes

Ans: $A\bar{C}\bar{D} + \bar{B}\bar{C} + BC + C\bar{D}$

(A=1 for both rows, so it remains)
(B=1 for one row & B=0 for another row, so it cancel)
(C & D=0 for both rows, so they remains as $\bar{C}$ & $\bar{D}$)